Where Forks Meet and Join: Lattices of Union-Finds

ANONYMOUS AUTHOR(S)

Automated provers reason about equality under many different assumptions, which induce a family of related equivalence relations over the same terms. Proof search deliberately explores such assumption contexts independently across proof subgoals and candidate theorems, but later proof steps often need to combine the equalities discovered under those separate contexts. The *union-find* is the underlying data structure that maintains an equivalence relation. Existing extensions to union-finds, and e-graphs built on top, support different equivalence relations over the same terms, called *colors* or *versions*, but do not support composing the equivalence relations induced by independently derived contexts without replaying work or duplicating relations.

In this paper, we show how to compose such equivalence relations efficiently. Based on the observation that equivalence relations form a lattice, we introduce two operations: combining relations by the equivalence closure of their union (their meet), and extracting the equalities common across multiple relations by intersection (their join). Practically, unions merge discovered facts, such as equalities derived while proving different theorems, whereas intersections extract facts that hold across all cases of a case split. We present a lazy extension of *versioned union-finds* that realizes both composition operations (the version meet and join), alongside the existing operation for branching from an equivalence relation (version fork). We provide proofs of correctness and complexity for the operations. Finally, we evaluate the operations and show that their cost scales as roughly $n^{1.5}$ on average against n^2 for the eager variants, falling back to n^2 only in the worst case, with at most 12% runtime overhead.

CCS Concepts: • **Theory of computation** → *Equational logic and rewriting*; *Program verification*; • **Computing methodologies** → *Theorem proving algorithms*.

Additional Key Words and Phrases: Automated Theorem Proving, Union-Find, E-Graphs, Conditional Reasoning

ACM Reference Format:
Anonymous Author(s). 2026. Where Forks Meet and Join: Lattices of Union-Finds. 1, 1 (July 2026), 23 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Automated theorem provers and SMT (Satisfiability Modulo Theories) solvers are widely used to verify the correctness of programs [Lattuada et al. 2023; Leino 2010], hardware designs [Coward et al. 2023], and mathematical statements [The mathlib Community 2020]. At their core, these tools discover and maintain equalities between terms and decide whether two terms are equal, e.g., to determine that a proof goal has been discharged or that a formula is satisfiable. Crucially, such equalities rarely hold unconditionally: theorem provers organize proofs into cases and explore one hypothesis at a time, and SMT solvers based on the DPLL(T) architecture [Nieuwenhuis et al. 2006] interleave a SAT solver, which drives case splits over the boolean structure of a formula, with theory solvers that decide equalities over uninterpreted functions [Nelson and Oppen 1980], e.g., in Z3 [de Moura and Bjørner 2008] and cvc5 [Barbosa et al. 2022]. In both settings, an equality that

ACM XXXX-XXXX/2026/7-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

holds in one branch of a case analysis, or under one assignment of the SAT solver, does not need to hold in another. A prover must therefore track many, only partially overlapping, assumption contexts at once. Each such context induces an equivalence relation over essentially the same set of terms, and these equivalence relations are related by the assumptions under which they were derived.

The data structure of choice for discovering and maintaining equalities is the *e-graph* [Nelson 1980; Willsey et al. 2021], which compactly represents a set of terms together with a congruence relation over them: whenever two terms are equal, applying the same function to both produces terms that are equal as well. Internally, e-graphs are built on top of a *union-find* [Galler and Fisher 1964; Tarjan 1975], a data structure that maintains a partition of elements into equivalence classes at near-constant amortized cost per operation [Hopcroft and Ullman 1973]. While the e-graph, and the provers and SMT solvers built on top of it, are the applications that motivate our operations, we will focus on the union-find layer, where equalities are actually stored and canonicalized.

Beyond SMT solving, e-graphs also serve as the core reasoning engine of automated inductive theorem provers and theory-exploration tools, such as Propel [Zakhour et al. 2023, 2024a], TheSy [Singher and Itzhaky 2021], and CCLemma [Kurashige et al. 2024].

A key limitation, however, lies in the union-find at their core: it maintains a single, global equivalence relation, which the e-graph inherits and which is a poor fit for conditional reasoning. Encoding the equalities of n proof branches then requires either cloning the structure n times, wasting memory and time on the equalities shared by all branches, or using an extension that maintains several equivalence relations over the same terms. Existing extensions do so with *colors* [Singher and Shachar 2024] or *versions* [Cesario et al. 2026], both maintaining multiple equivalence relations on top of a single, shared term base; we describe versioned e-graphs here and compare with colored e-graphs in Section 5. The key insight of versioned e-graphs is that the equivalence relations induced by assumption contexts are not independent: a relation derived under stronger assumptions should inherit the equalities known under weaker assumptions. Versions therefore form a *version tree*: a *fork* operation creates a child version that inherits all the equalities of its parent, while any further equality asserted in the child is visible only to the child and its descendants. The underlying *versioned union-find* implements this tree of equivalence relations lazily, reifying an inherited equality only when it is actually queried, so that the cost of inheriting a parent's equalities is amortized across queries instead of being paid upfront at fork time.

Tree-shaped inheritance captures only one way in which induced equivalence relations are related. In a prover, assumption contexts are often explored independently because proof search factors work across proof subgoals, case splits, and candidate theorems. At the same time, later proof steps often span those subproofs, and their equivalence relations need to be composed. For example, a rewrite may become applicable only when two independently established hypotheses are available together, equalities derived while proving one candidate theorem may be useful while proving another and should be merged into the shared search state, and a parent goal may need exactly the facts that hold across all cases of a case split. These situations require constructing new equivalence relations from several existing ones, not just from a single parent relation.

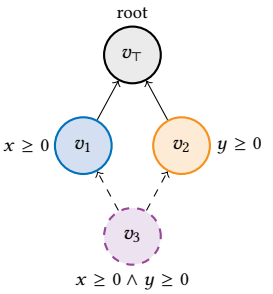


Fig. 1. The composed version v_3 depends on both v_1 and v_2, which the version *tree* cannot express.

As a running example, consider a prover that establishes that the absolute value acts as the identity on products of non-negative operands, i.e., that $x \geq 0$ and $y \geq 0$ imply $|x \cdot y| = x \cdot y$, assuming *abs* is multiplicative, i.e., $|x \cdot y| = |x| \cdot |y|$:

Listing 1. Running example. Definition of $|x|$.

```
fun abs(x) =
  match x >= 0
   case True: x
   case False: -x
```

The prover explores $abs(x)$ (fork v_1 with hypothesis $x \geq 0$) and $abs(y)$ (fork v_2 with hypothesis $y \geq 0$) independently, e.g., for parallelism. Both forks stem directly from the base version, one for each operand. The interesting case is the one where both hold, i.e., $x \geq 0$ and $y \geq 0$, since only then does the property's premise hold and the prover must show $abs(x) \cdot abs(y) = x \cdot y$. Checking this sub-goal, however, requires a version v_3 that holds *both* the hypothesis recorded by v_1 ($x \geq 0$) and the one recorded by v_2 ($y \geq 0$) simultaneously (Figure 1). Since, in current approaches, every version has a single parent, v_3 can only be forked from v_1 or from v_2, never from both, even though it semantically depends on the pair: whichever hypothesis is not inherited from the chosen parent must then either be replayed inside v_3 (rerunning work that the versioned union-find has already performed and stored for the sibling version) or be eagerly copied into a new materialized relation. The issue is therefore not that the desired equivalence relation is impossible to represent, but that existing techniques do not support efficient composition of relations from independent contexts while preserving sharing.

In this paper, we trace this limitation to the algebraic structure of the equivalence relations induced by assumption contexts. Versions denote equivalence relations over a common term set. These relations form a non-distributive lattice. This is the equality-side presentation of the partition lattice studied in partition logic [Ellerman 2010]: partition logic reasons with the distinctions a partition makes, whereas union-finds represent the equalities it keeps.

Forking versions as in existing approaches exposes only the tree-shaped fragment generated by single-parent coarsening. The full lattice also contains operations that combine arbitrary induced relations: the equivalence closure of their union, which aggregates equalities discovered in any dependency, and their intersection, which keeps exactly the equalities common to all dependencies. Orienting the version lattice by coarsening, as versioned union-finds do, we call the first the *meet* and the second the *join* of versions[1]: the meet is coarser than either dependency and the join finer than both.

Guided by this lattice structure, we generalize the *version tree* into a *version DAG (directed acyclic graph)* that supports these dependencies directly, and we propose two algorithms that lazily compute them on top of a union-find. We prove both algorithms correct and analyze their complexity, and we show, through an implementation, that their cost scales as roughly $n^{1.5}$ on average against n^2 for an eager implementation of the same operations, at the cost of a slightly worse worst case for runtime. We also examine how both operations extend to versioned e-graphs: the meet preserves congruence by construction, whereas the join is *relatively complete*, exact over a chosen finite set of important terms (Section 3.4).

In summary, this paper makes the following contributions.

- We identify the lattice structure of the equivalence relations denoted by versions in versioned union-finds and relate it to the equality-side presentation of the partition lattice studied by partition logic (Section 2.3).
- Guided by this structure, we generalize the version tree into a *version DAG*, exposing the composition operations that the existing tree-based approaches cannot represent directly (Section 3.1).

[1]This is dual to the refinement order commonly used for equivalence relations, under which the equivalence closure of the union is the *join*. We orient the lattice by coarsening throughout, matching how versioned union-finds grow downward from the identity relation (Section 2.3).

- We propose an algorithm that lazily determines, on top of a union-find, the equivalence relation of a version that computes the *meet* of its dependencies, i.e., the equivalence closure of their union, and we prove it correct and analyze its time and space complexity (Section 3.2).
- We propose an algorithm that lazily determines the *join* of a version's dependencies, i.e., their intersection, and we prove it correct and analyze its time and space complexity (Section 3.3).
- We implement the meet and join and evaluate them against eager variants: on average, the lazy operations perform up to 94% fewer operations and use up to 93% less memory, while in the worst case they match the eager variants with at most about 12% runtime overhead (Section 4).

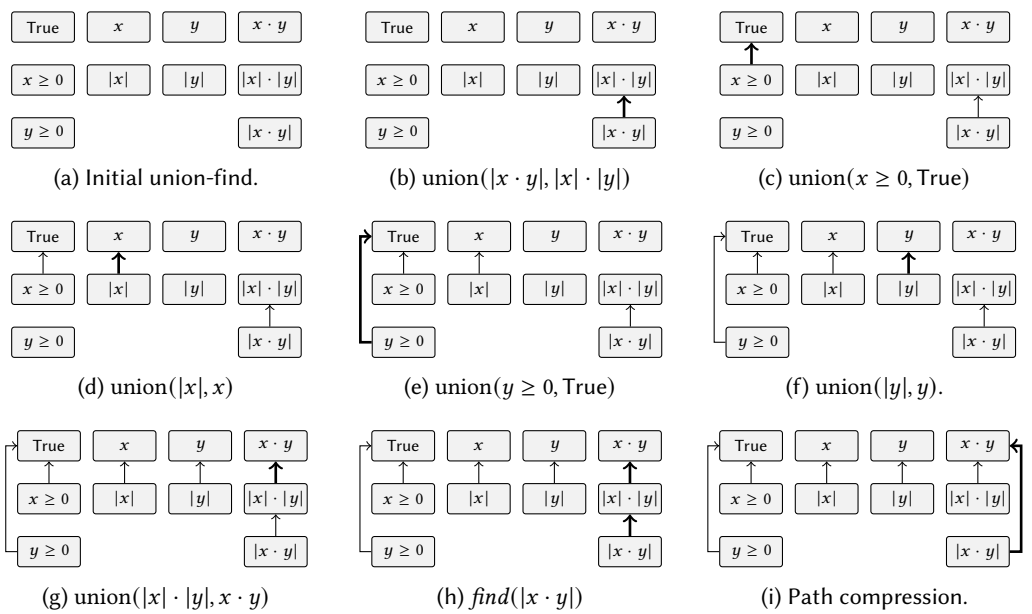
2 Background

In this section, we introduce the classical union-find algorithms and their versioned extension, which compactly represents multiple equivalence relations. Then, we describe the lattice of equivalence relations, its relationship to partition logic, and its connection to versioned union-finds, which motivates our extension of versioned union-finds with the meet and join operations.

2.1 Union-Finds

The union-find [Galler and Fisher 1964; Tarjan 1975] is a data structure that maintains an equivalence relation over a set of elements, i.e., a partition of the elements into equivalence classes. In particular, the union-find is a forest, whose nodes are elements, and equivalence is defined by connectivity in the forest: two elements are equal if they are connected by a path of edges. Given the forest structure, this definition can be further simplified: two elements are equal if they share the same root, which is called the *canonical* of their equivalence class (i.e., the root of their tree).

Starting from the trivial union-find, where each element is its own equivalence class, the union-find exposes two operations to build any equivalence relation: the *union* operation makes one



(a) Initial union-find. (b) union($|x \cdot y|, |x| \cdot |y|$) (c) union($x \geq 0$, True)

(d) union($|x|, x$) (e) union($y \geq 0$, True) (f) union($|y|, y$).

(g) union($|x| \cdot |y|, x \cdot y$) (h) *find*($|x \cdot y|$) (i) Path compression.

Fig. 2. The union-find for our running example (Listing 1). We highlight changes in bold.

root the parent of another, merging their equivalence classes; the *find* operation traverses parent pointers from an element to its root, returning the canonical of its equivalence class. We will discuss further details for these operations later in Listing 2, when we will compare them with their versioned variants.

In Figure 2, we build a union-find resulting from the analysis of our running example (Listing 1). For simplicity, we start from an initial union-find where all elements have already been added (2a), i.e., all terms have been pre-collected. There, all elements are canonicals (usually represented as a self-referencing parent pointer, that we omit). First, the prover decides to simplify the goal by applying multiplicativity of *abs* (2b): note how the union operation always connects canonicals, which can be ensured by using the find operation. Then, it starts exploring the cases for $x \geq 0$ and $y \geq 0$ *in sequence* (2c-2d and 2e-2f respectively). Finally, the prover completes the proof by congruence of multiplication (2g). In the end, the union-find represents the equivalence relation between all programs in the proof space. For example, we can query the canonical of $|x \cdot y|$ using find and discover it to be $x \cdot y$ (2h). Then, path compression can shorten the path traversed for future queries (2i).

2.2 Versioned Union-Finds

The *versioned union-find* [Cesario et al. 2026] is a data structure that maintains a family of equivalence relations (or *versions*) over a single, shared set of elements, i.e., one partition of the elements for each *version*, with versions organized into a *version tree*. In particular, the versioned union-find is a graph whose nodes are elements and whose edges are labelled by version, and equivalence in a version is defined by connectivity through the edges of that version or its ancestors (*valid edges*): two elements are equal in a version if they are connected by such a path. Canonicalization is also preserved: two elements are equal in a version if they share the same root along valid version's edges, which is the *canonical* of their equivalence class in that version.

The version tree orders versions by *coarsening*: a child version retains all the equalities of its parent and possibly adds more, hence induces a coarser partition of the terms. The *root version* is the top element of the lattice, representing the identity relation, and it's always present. A *fork* operation creates a new child version from a parent version, inheriting all of its equalities. In Figure 3, we show the version tree for the running example, defining the equivalence relations for the root version ($v_\top$), for $x \geq 0$ (v_1) and $y \geq 0$ (v_2). There, $v_\top$ and v_1 are dependent versions: in particular, v_1 coarsens $v_\top$ or equivalently $v_1 \prec v_\top$. Instead, v_1 and v_2 are independent versions: they are not comparable in the lattice, i.e., neither $v_1 \prec v_2$ nor $v_2 \prec v_1$. Because they are independent, a prover can reason about them independently, e.g., in parallel.

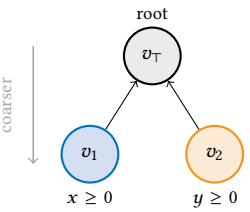


Fig. 3. The version tree for the running example.

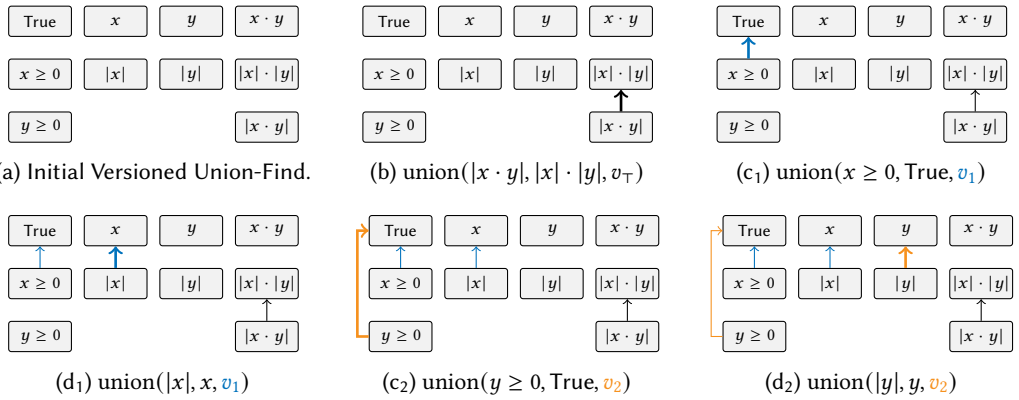
Listing 2 describes the versioned union-find algorithm as an extension of the classical union-find operations, highlighting the differences in green. Like the classical union-find, the versioned union-find exposes a *find* and a *union* operation, except that both are now indexed by the version in which they operate. The *find* operation (line 1) traverses parent pointers from an element to the root of its tree in a version v, returning the canonical of its equivalence class in v; the *union* (line 7) operation makes the canonical of one element the parent of the canonical of the other, merging their equivalence classes in v.

The first difference is that an element does not need to carry an edge for every version. When *find* looks up the parent of an element in v (line 2), it does not read a single pointer but queries the *closest valid parent* (*cvp* at line 21): if the element has a reified edge in v it is returned, and otherwise

Listing 2. Versioned Union-Find Algorithm.

```
1  fun find(x: Node, v: Version): Node =
2    p = cvp(x, v) // closest valid parent
3    x' = if p != x then find(p, v) else x
4    add_edge(x', x, v) // path compression
5    x'
6
7  fun union(x: Node, y: Node, v: Version) =
8    x' = find(x, v)
9    y' = find(y, v)
10   for sv in v.children do
11     union(x, y, sv)
12   add_edge(x', y', v)
```

```
13 fun add_node(e: Element): Node =
14   x = Node(e)
15   add_edge(x, x, Version.Root)
16   x
17
18 fun add_edge(p: Node, c: Node, v: Version) =
19   c.parent[v] = p
20
21 fun cvp(x: Node, v: Version): Node =
22   if v in x.parent
23   then x.parent[v]
24   else cvp(x, v.parent)
```



(a) Initial Versioned Union-Find. (b) union($|x \cdot y|, |x| \cdot |y|, v_\top$) ($c_1$) union($x \geq 0$, True, v_1)

(d_1) union($|x|, x, v_1$) (c_2) union($y \geq 0$, True, v_2) (d_2) union($|y|, y, v_2$)

Fig. 4. The versioned union-find for our running example (Listing 1). We highlight changes in bold.

the search walks up the version tree to the closest ancestor version that does (line 24), terminating at the root version's self-edge. Path compression can also be applied by adding a new edge in v from the element to the canonical of its equivalence class (line 4): when active, a forked version will lazily clone its parent edges when they are first queried, trading memory for runtime. The second difference is that an equality asserted in v must also hold in every version that coarsens it. Hence *union* propagates the merge downward, recursively unioning the two elements in each child version (lines 10 to 11) before adding the edge in v itself (line 12). Propagation can be avoided by working only on leaf versions.

Figure 4 constructs the versioned union-find for our running example step by step, mirroring the operations of Figure 2 but now recording each edge in the version that owns it. As before, we start from an initial versioned union-find where all terms have been pre-collected and every element is its own canonical in the root version (4a). The prover first simplifies the goal in the shared root version $v_\top$ by multiplicativity of *abs* (4b); since this equality is owned by the root version, it will be inherited by every descendant. It then decides to explore the two guards in parallel, one in each child of the root version: v_1 assumes $x \geq 0$ ($4c_1$) and records $|x| = x$ ($4d_1$), while v_2 assumes $y \geq 0$ ($4c_2$) and records $|y| = y$ ($4d_2$). Because the two branches are independent, they can be explored concurrently, and each child sees its own edges layered on top of the root version's, but not its sibling's: v_1 establishes $|x| = x$ and v_2 establishes $|y| = y$, yet neither equality is visible in the other. As a result, neither child alone can discharge the goal $|x \cdot y| = x \cdot y$: v_1 can prove $|x \cdot y| = x \cdot |y|$, and symmetrically for v_2. Completing the proof requires a version that holds both guards at once. The prover can solve this by merging the two equivalence relations in a new version, but this is

inefficient in general: consider a new guard $z \geq 0$ that implies $x < 0$, then exploring $x \geq 0 \wedge y \geq 0$ and $z \geq 0 \wedge y \geq 0$ independently will require duplicating the information of $y \geq 0$ in both branches.

2.3 Partition Logic and Equivalence Relations

The equivalence relations stored by a union-find are also partitions of the underlying set of terms. The lattice of such partitions is closely related to partition logic [Ellerman 2010], but our algorithms use the equivalence-relation presentation natural for union-find. In Ellerman's terminology, partition logic reasons with the *distinctions* a partition makes, i.e., pairs of elements it separates. Union-finds instead represent the dual *indistinctions*, i.e., the equalities or equivalence-relation pairs kept in the same block. Table 1 compares the Boolean lattice of subsets, the distinction-side partition lattice, and the equality-side equivalence-relation lattice used in this paper.

Propositional logic can be interpreted as the logic of *subsets* of a fixed universe U: a proposition is identified with the set of elements of U at which it holds, and is characterized by the *elements* it contains: each element is simply in or out. Conjunction, disjunction, and negation are intersection, union, and complement, and ordering propositions by inclusion ($A \preceq B$ iff $A \subseteq B$, i.e. A *implies* B) yields a Boolean lattice whose bottom is the empty set and whose top is all of U. This lattice is *distributive* and *uniquely complemented* (every proposition has exactly one negation), and it is the well-behaved baseline against which Table 1 sets the partition and equivalence-relation lattices.

Partition logic is about the *partitions* of U: divisions of the universe into disjoint, exhaustive *blocks*. While a subset is determined by the elements it contains, a partition is characterized by the pairs it separates. We call such a separated pair a *distinction* and write $x \nsim y$ when x and y fall in different blocks of the partition. Ordering partitions by inclusion of their distinctions, the single-block partition (no distinctions) is the bottom and the all-singletons partition (every distinction) is the top. The meet of two partitions merges any blocks sharing some element, and the join keeps two elements together only when both do. This lattice is *not* distributive and its complements are not unique [Grätzer 2011; Ore 1942], so negation and implication no longer behave classically.

A partition is also an *equivalence relation*: its blocks are the equivalence classes, and the pairs it keeps together are its *equalities* $x \sim y$. The third column of Table 1 presents this same partition lattice through equalities rather than distinctions, ordering relations by inclusion of their equality pairs ($E_1 \preceq E_2$ iff E_1 *refines* E_2). Because equalities and distinctions are complementary, this order is dual to the distinction order: the identity relation (every element alone) is the bottom, the total relation (one block, all elements equal) is the top, meet is intersection, and join is the equivalence closure of union.

These equivalence relations are exactly the objects represented by versions in a versioned union-find: a version denotes the equalities it has proved, and its blocks are the equivalence classes shown in Figure 4. The rest of the paper uses this equality-side presentation, with the orientation reversed

Table 1. Propositional, partition, and equivalence logic over one fixed universe U. tc is transitive closure.

	Propositional Logic	Logic of partitions	Logic of equivalence relations
Object	subset $A \subseteq U$	partition π of U	equivalence relation E on U
Primitive	element $x \in A$	distinction $x \nsim y$	equality $x \sim y$
Order $\preceq$	inclusion of elements	inclusion of distinctions	inclusion of equalities
Bottom $\bot$	$\emptyset$	$\{U\}$	id
Top $\top$	U	$\{\{a\} \mid a \in U\}$	U^2
Meet $\wedge$	$A \cap B$	$\mathrm{tc}(\pi_1 \cup \pi_2)$	$E_1 \cap E_2$
Join $\vee$	$A \cup B$	$\pi_1 \cap \pi_2$	$\mathrm{tc}(E_1 \cup E_2)$

Table 2. Laws of the Boolean lattice of subsets and the version lattice (✓ holds, ✗ fails).

Property	Definition	Condition	Version
Meet $\wedge$ *and join* $\vee$			
Idempotence	$a \wedge a = a$	✓	✓
Commutativity	$a \wedge b = b \wedge a$	✓	✓
Associativity	$a \wedge (b \wedge c) = (a \wedge b) \wedge c$	✓	✓
Absorption	$a \wedge (a \vee b) = a$	✓	✓
Identity	$a \wedge \top = a,\ a \vee \bot = a$	✓	✓
Distributivity	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	✓	✗
Modularity	$a \preceq c \Rightarrow a \vee (b \wedge c) = (a \vee b) \wedge c$	✓	✗
Complement $\neg$			
Existence	$\exists a'.\ a \wedge a' = \bot,\ a \vee a' = \top$	✓	✓
Uniqueness	such a' is unique	✓	✗
Involution	$\neg\neg a = a$	✓	✗
De Morgan	$\neg(a \wedge b) = \neg a \vee \neg b$	✓	✗
Implication $\rightarrow$			
Unit test	$a \rightarrow b = \top \iff a \preceq b$	✓	✓
Definability	$a \rightarrow b = \neg a \vee b$	✓	✗
Residuation	$a \wedge x \preceq b \iff x \preceq a \rightarrow b$	✓	✗

when convenient to match versioned union-finds: the root version is drawn at the top, and adding equalities moves downward to coarser versions. This reversal swaps the names of meet and join relative to equality-pair inclusion, but not the underlying operations or laws.

3 Meet and Join in Versioned Union-Finds

In this section, we interpret versioned union-finds as a data structure presentation of the version lattice, then derive lazy algorithms for the lattice operations that tree-shaped versions leave implicit.

3.1 Version Lattice

The first contribution of this paper is to identify the structure of the versions in a versioned union-find as an instance of the equivalence-relation lattice.

Concretely, the terms of the prover form the universe U, and each version is one equivalence relation over them, namely the equalities it has proved. The versions of Section 2.2 are therefore precisely the equivalence relations of Section 2.3, and the version lattice is the same lattice shown in the equivalence-relation column of Table 1. It differs in one respect only: versioned union-finds orient it by *coarsening* rather than by refinement. That is, $u \preceq w$ holds when u is coarser than w, meaning that u can prove every equality of w and possibly more. This orientation places the identity relation at the top, the root version $v_\top$ that has proved nothing, and each coarser version below it, exactly matching the version tree of Figure 3. A fork assumes an additional hypothesis and thereby coarsens, descending from a parent to a child, so the tree is precisely the fragment of the lattice reachable by such single-parent descents. What the tree cannot express, and the lattice supplies, is the relationship between independent versions such as the sibling forks v_1 and v_2, which is recovered through their meet and join.

By connecting the version lattice to the equivalence-relation lattice, we recover its laws, which Table 2 lays out against the Boolean lattice of subsets. Both lattices agree on the core lattice laws: for both, meet and join are idempotent, commutative, associative, and mutually absorptive, and each operation has an identity, the top $v_\top$ for meet and the bottom $v_\bot$, the total relation in which

Listing 3. Syntax of versions.

```
data Version
  = Root
  | Fork { parent: Version }
  | Meet { deps: List[Version] }
  | Join { deps: List[Version] }
```

Listing 4. The traversal logic of find is version-dependent.

```
fun find(x: Node, v: Version): Node =
  p = if v in x.parent then x.parent[v] else v.traverse(x) // version-dependent
  x0 = if p == x then x else find(p, v)
  add_edge(x0, x, v)          // path compression
  x0

fun fork.traverse(x: Node): Node =
  find(x, fork.parent)
```

all terms are equal, for join. The divergence begins with the laws that make the Boolean lattice well behaved. The version lattice is non-distributive, and even the weaker modular law fails, so a prover cannot in general push a meet across a join. Complements still exist, as every version v admits some v' with $v \wedge v' = v_\bot$ and $v \vee v' = v_\top$, but they are no longer unique, no longer involutive, and De Morgan's laws break. The order still captures entailment, but implication cannot be defined as $\neg a \vee b$ nor as a residuated connective. The version lattice therefore offers no canonical negation and no implication connective. In short, versions may be combined by meet and join and ordered by entailment exactly as subsets are, but the Boolean reasoning a prover applies to its conditions does not carry over to the versions those conditions induce.

To adapt the versioned union-find, we can represent each version in the lattice by the way it is built (Listing 3). A *root* is the identity relation $v_\top$, the top of the lattice, always present. A *fork* stores a single parent and coarsens it with further equalities, exactly the versions of Section 2.2 that the version tree already captures. The two lattice operations supply what the tree cannot: a *meet* and a *join* each store a list of dependencies and denote, respectively, the meet ($\wedge$) and the join ($\vee$) of their dependencies' relations. Because a meet or a join may name several dependencies, versions form not a tree but a *directed acyclic graph*, in which each version points to the versions it is built from.

These laws shape the representation and the algorithms that follow. Idempotence, commutativity, and associativity make an n-ary meet or join well-defined regardless of the order and multiplicity of its dependencies, which is what lets a version store its dependencies as an unordered list and lets the DAG share common sub-versions. Absorption also identifies which dependencies are semantically redundant when the order between them is known: in a meet, the finer of two comparable dependencies does not change the result, since the coarser one already proves its equalities, and in a join, the coarser one does not change the result, since the finer dependency already removes those extra equalities. While such dependencies could be dropped, our algorithms do not rely on detecting such redundancies and compute the same meet or join for arbitrary dependency lists. Because the lattice is non-distributive and not even modular, however, there is no Boolean-style normal form that can in general factor arbitrary compositions to share work. We therefore compute each composition against the represented relations themselves, lazily and on demand. Finally, because complements are not De Morgan dual, the join cannot be recovered from the meet by complementation as it would be in a Boolean setting, so the meet (Section 3.2) and the join (Section 3.3) require separate algorithms.

These laws shape the representation and the algorithms that follow. Idempotence, commutativity, and associativity make an n-ary meet or join well-defined regardless of the order and multiplicity

of its dependencies, which is what lets a version store its dependencies as an unordered list and lets the DAG share common sub-versions. Absorption also identifies which dependencies are semantically redundant when the order between them is known: in a meet, the finer of two comparable dependencies does not change the result, since the coarser one already proves its equalities, and in a join, the coarser one does not change the result, since the finer dependency already removes those extra equalities. While such dependencies could be dropped, our algorithms do not rely on detecting such redundancies and compute the same meet or join for arbitrary dependency lists. Because the lattice is non-distributive and not even modular, however, there is no Boolean-style normal form that can in general factor arbitrary compositions to share work. We therefore compute each composition against the represented relations themselves, lazily and on demand. Finally, because complements are not De Morgan dual, the join cannot be recovered from the meet by complementation as it would be in a Boolean setting, so the meet (Section 3.2) and the join (Section 3.3) require separate algorithms.

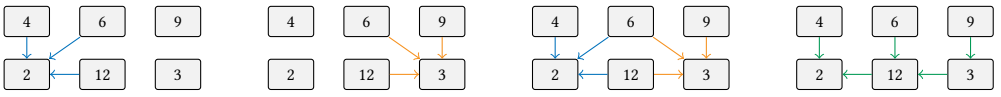
With the syntax in place, we adapt the versioned *find* so that its traversal depends on the kind of version (Listing 4). Every kind shares one skeleton: to canonicalize x in a version v, *find* follows x's reified edge in v when it has one, and otherwise defers to a version-specific *traverse*. Then, it walks up to the root and path-compresses by reifying an edge from x to that root, just as the versioned union-find does. Only the *traverse* varies with the kind. A root needs none: every element carries a self-edge and is already its own canonical. A fork delegates to its parent, recovering the closest-valid-parent traversal of Listing 2. A meet and a join instead explore their dependencies, whose traversals we develop in the following subsections.

3.2 The Meet of Versions

The meet $v_1 \wedge v_2$ combines two versions by taking the transitive closure of the union of their equivalence relations: two terms are equal in the meet whenever a chain of equalities, each step proved by *either* v_1 or v_2, links them. A versioned union-find represents every version as a forest of edges, so we can read the meet off directly. Overlaying the two forests yields a graph, and the equivalence classes of the meet are exactly its connected components. Because a chain may alternate freely between the two versions, the meet proves every equality that either input proves and possibly more, so it is coarser than both and sits below them in the version lattice.

Figure 5 illustrates the meet over the numbers $\{2, 3, 4, 6, 9, 12\}$. Version v_1 equates the multiples of 2 and version v_2 the multiples of 3, each collapsing its multiples to a single canonical (Figures 5a and 5b). The two classes share the multiples of 6, namely 6 and 12, so overlaying the forests (Figure 5c) links them into a single connected component: the meet equates every number divisible by 2 or by 3, whereas any number coprime to 6 would instead stay in its own block. No single version equates 4 and 9, but the meet does by bridging through 6: 4 and 6 are equal in v_1, and 6 and 9 are equal in v_2. The overlay is a graph rather than a forest, since a shared multiple of 6 has a parent in each version; to recover a union-find we fix an arbitrary order on the elements, e.g., take the least element of each block as its canonical, reifying the block as a single tree rooted at 2 (Figure 5d). This difficulty is instructive, as it highlights that a union-find is concerned not only with maintaining equivalences (Figure 5c), but also with canonicalization (Figure 5d): the harder problem of consistently electing a single representative for each class.

Listing 5 eagerly computes the meet of two classical union-finds, assuming they share the same universe. Starting from the identity union-find over the shared elements, in which every element is its own canonical (line 2), it walks the elements and unions each with its canonical in v_1 and then in v_2 (lines 4 to 5). This replays both equivalence relations into a single structure. Moreover, the result is a union-find by construction, ensuring canonicalization by some arbitrary ordering. The algorithm is eager: it performs a constant number of union and find operations per element, materializing



(a) v_1: $2k$. (b) v_2: $3k$. (c) v_1, v_2: overlaid. (d) $v_1 \wedge v_2$: $2k$ or $3k$.

Fig. 5. The meet of two versions, quotienting the natural numbers by divisibility by 2 and 3.

Listing 5. Eager computation of the meet.

```
fun meet(v1: UnionFind, v2: UnionFind): UnionFind =
  m = UnionFind() // identity
  for x in universe do
    m.union(x, find(x, v1)) // replay v1
    m.union(x, find(x, v2)) // replay v2
  m
```

Listing 6. Lazy computation of the meet.

```
1  fun meet.traverse(x: Node): Node =
2    // explore the connected component of x across all dependencies
3    x0 = x; touched = {x}; queue = [x]
4    while queue not empty do
5      a = queue.dequeue()
6      if meet in a.parent then // short-circuit
7        x0 = find(a, meet)
8        break
9      for v in meet.deps, b in neighbors(a, v) do
10       if b not in touched then
11         touched.add(b); queue.enqueue(b)
12   for y in touched do add_edge(x0, y, meet) // reify
13   return x0
```

the whole meet up front and paying for every element even when only a few canonicals are ever queried, resulting in a complexity of $O(n\alpha(n))$ for n elements in the universe, where α is the inverse Ackermann function.

In the versioned union-find, just as a fork does not copy its parent's equalities but lets the versioned *find* reify them lazily, we want the meet to avoid materializing its versioned edges up front. Creating a meet version should be cheap, recording only its dependencies, and the work of Listing 5 should be deferred to *find*, paid only for the elements that are actually queried and amortized across queries. The difference is that a fork inherits from a single ancestor, so its *find* delegates up the version tree through the closest valid parent; the meet inherits from several dependencies at once, whose equalities are connectivity in the union of their forests, so *find* cannot follow one parent but must, on demand, consult the dependencies and reify just enough meet edges to canonicalize the queried element consistently.

A meet version therefore stores only its dependencies, and supplies the *traverse* (Listing 6) that the generic *find* (Listing 4) invokes when x has no reified meet edge. It computes the canonical of x on demand by a breadth-first search from x over the elements reachable through the edges of any dependency (lines 3 to 9). That connected component is x's equivalence class in the meet.

The search takes the queried element x as the provisional canonical (line 3) and stops early if it reaches an element that already carries a meet edge: that element's meet canonical is the canonical of the whole component, and the search adopts it (line 6). Otherwise the component has no reified canonical yet, and x itself is arbitrarily[2] chosen as the canonical. Either way, the search ends with

[2]A specific canonical order, such as the least-element rooting of Figure 5d, could also be enforced after the search.

path compression, reifying a meet edge from every touched element to this canonical (line 12), so that later queries into the same component are served directly by the generic *find* without re-entering the traverse. In Figure 5, querying 9 walks the block $\{2, 3, 4, 6, 9, 12\}$ and, finding no reified canonical, keeps 9 as the canonical and reifies a meet edge from each of these elements to 9; a subsequent query of any of them then returns 9 directly.

Both composition operations maintain the same structural invariant, which we name once and reuse for the join in Section 3.3.

Definition 1 (Rooted-Class Invariant). After any sequence of *find*s on a composition version m, every reified edge of m connects an element to an element of the same class of m's denoted relation, and every class of that relation contains at most one root among the elements reified in m.

Theorem 1 (Meet Equivalence). *Let $m = v_1 \wedge v_2$ be a meet. The traverse of x reifies exactly the elements meet-equal to x to a common canonical, where two elements are meet-equal when they are connected in the overlay graph whose edges are those of the two dependency forests together. The reified edges thus represent the meet relation $\mathrm{tc}(E_1 \cup E_2)$, the equivalence closure of the union of the relations E_1, E_2 of v_1, v_2.*

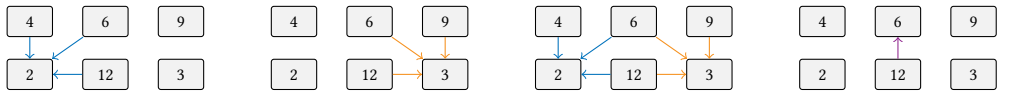
Proof. Two elements x and y are meet-equal exactly when a chain of equalities, each proved by v_1 or by v_2, links them, which is exactly connectivity in the overlay graph. The traverse's breadth-first search from x follows exactly those edges (line 9), so the component it reaches is x's meet class; it then reifies a meet edge from every element of that component to one canonical (line 12). □

Theorem 2 (Meet Canonicalization). *Over any sequence of finds, a meet $m = v_1 \wedge v_2$ maintains the Rooted-Class Invariant (Definition 1): every meet class has a single root. Hence $find(x, m) = find(y, m)$ if and only if x and y are meet-equal.*

Proof. A component is always rooted at a single element. A query whose search meets no reified edge roots the whole component at the queried element x, reifying every touched element to it; a later query into that component then finds a reified edge and is served by the generic *find* (Listing 4) without re-entering the traverse. A query whose search does reach a reified element instead adopts that element's canonical (line 6) and attaches the newly touched elements to the existing tree rather than starting a competing one. Either way no two meet-equal elements end up under different roots, so *find* decides meet-equality correctly. □

Proposition 1 (Meet Complexity). *Let $m = v_1 \wedge v_2$ be a meet over n elements with dependencies held fixed, as the eager meet also assumes. Any sequence of finds on m that reifies $R \leq n$ edges runs in $O(R\,\alpha(n))$ time. Hence the meet costs $O(1)$ when it is never queried ($R = 0$) and $O(n\,\alpha(n))$ in the worst case ($R = n$), matching the eager meet (Listing 5).*

Proof. We charge every unit of work to a reified element; R counts the elements ever reified in m, i.e., the queried elements together with the components the traverse drags in. Creating m reifies no edge, so it costs $O(1)$, and once an element is reified it is served by the generic *find* and never re-enters the traverse; so all cost is incurred by the *find* that first reifies an element, which explores x's component in the overlay of the two dependency forests. An already reified element bounds the search rather than being expanded (line 6), so each element is expanded at most once over the whole run, and a component of c elements costs $O(c)$ neighbor steps and $O(c)$ reifications. The cost is therefore $O(\alpha(n))$ amortized per reified element, and summing over the R of them gives $O(R\,\alpha(n))$. □



(a) v_1: $2k$. (b) v_2: $3k$. (c) v_1, v_2: overlaid. (d) $v_1 \vee v_2$: $2k$ and $3k$.

Fig. 6. The join of two versions, quotienting the natural numbers by divisibility by 2 and 3.

Listing 7. Eager computation of the join.

```
1  fun join(v1: UnionFind, v2: UnionFind): UnionFind =
2    j = UnionFind()
3    reps = map()   // product -> representative
4    for x in universe do
5      p = (find(x, v1), find(x, v2)) // product of canonicals
6      j.union(x, reps.get_or_insert(p, x)) // join equal products
7    j
```

Listing 8. Lazy computation of the join.

```
1  fun join.traverse(x: Node): Node =
2    // group the smallest dependency class by product
3    product = fun a. (find(a, v) for v in join.deps)
4    d0 = smallest(x, join.deps) // dependency with x's smallest class
5    reps = map(); touched = []
6    for a in members(x, d0) do
7      if join in a.parent then reps[product(a)] = find(a, join)
8      else touched.append(a)
9    for y in touched do
10     y0 = reps.get_or_insert(product(y), y)
11     add_edge(y0, y, join)
12   return reps[product(x)]
```

3.3 The Join of Versions

The join $v_1 \vee v_2$ *intersects* the two equivalence relations: two terms are equal in the join only when they are equal in *both* v_1 and v_2. A term's canonical in the join is thus the tuple of its canonicals across the dependencies, its *product*, and two terms are equal in the join when their products are equal. Note, however, that the product is not an element of any forest: it can be used to decide equality but does not elect a canonical for the classes it defines. So, as before, we will apply an arbitrary ordering to elect those canonicals.

Figure 6 shows the join over the versions of the previous example, i.e., the multiples of 2 and of 3. Intersecting the relations keeps only the numbers that are multiples of *both*, the multiples of 6: 6 and 12 share the canonical product $(2, 3)$ and form the one nontrivial equivalence class, while every other number stays a singleton. In the end, if the meet fused the whole block $\{2, 3, 4, 6, 9, 12\}$, the join retains just $\{6, 12\}$.

The eager join has the same shape as the meet, but groups elements by their product instead of replaying edges. Listing 7 scans every element x of the universe and computes its product, that is, its canonical in each dependency (line 5). A map sends each product to a representative element: the first element seen with a given product becomes its representative, and every later element with the same product is unioned to it (line 6). Two elements therefore land in the same class exactly when their products coincide, which is exactly when they are equal in both dependencies. Like the meet, the algorithm is eager, at $O(n\,\alpha(n))$ cost[3].

[3]This assumes constant-time access to the product-to-representative map, as provided by a hash map.

Laziness is desirable here for the same reasons as the meet, so a join version likewise stores only its dependencies and supplies a *traverse* that *find* invokes when x has no reified join edge. The class of x is the set of elements sharing its product. Because those elements agree with x in both dependencies, they all lie in x's class in the smaller dependency[4], so the traverse enumerates that one class (line 6), groups its elements by product, and elects a representative for each product, reusing one already reified by an earlier query (line 10). It then reifies a join edge from every enumerated element to its product's representative (line 11) and returns x's. In Figure 6, querying 6 scans its class in the multiples of 3, $\{3, 6, 9, 12\}$, and finds that 6 and 12 share the product $(2, 3)$ and reify to a common representative, while 3 and 9, each alone in its product, stay singletons.

Theorem 3 (Join Equivalence). *Let $m = v_1 \vee v_2$ be a join. The traverse reifies two elements to a common representative exactly when their* products $(\mathit{find}(\cdot, v_1), \mathit{find}(\cdot, v_2))$ *coincide, which holds exactly when they are join-equal, i.e., equal in both v_1 and v_2. The reified edges thus represent the intersection $E_1 \cap E_2$ of the relations of v_1, v_2.*

Proof. Two elements x and y are join-equal exactly when they are equal in both dependencies. Because $\mathit{find}(\cdot, v)$ canonicalizes each equivalence class in v, this happens exactly when their products $(\mathit{find}(x, v_1), \mathit{find}(x, v_2))$ and $(\mathit{find}(y, v_1), \mathit{find}(y, v_2))$ coincide; the product therefore decides join-equality. The traverse elects one representative per product and reifies a join edge from every element to it, so two elements receive the same root exactly when their products coincide. □

Theorem 4 (Join Canonicalization). *Over any sequence of finds, a join $m = v_1 \vee v_2$ maintains the Rooted-Class Invariant (Definition 1): every join class has a single representative. Hence $\mathit{find}(x, m) = \mathit{find}(y, m)$ if and only if x and y are join-equal.*

Proof. Every element join-equal to x shares its class in both dependencies, hence in the smaller one (line 4), so enumerating that single class captures x's whole join class; grouping it by product separates the join classes it contains, and line 10 reuses the representative already reified for a product before electing a fresh one. Because every query looks a product up before choosing, all queries agree on its representative, and the edges reified for one query remain valid for later ones. □

Proposition 2 (Join Complexity). *Let $m = v_1 \vee v_2$ be a join over n elements with dependencies held fixed, as the eager join also assumes. Any sequence of finds on m that reifies $R \leq n$ edges runs in $O(R\,\alpha(n))$ time, where each query enumerates a single dependency class of size s at cost $O(s\,\alpha(n))$, bounded by the smaller dependency class rather than by the whole universe as in the meet. Hence the join costs $O(1)$ when it is never queried ($R = 0$) and $O(n\,\alpha(n))$ in the worst case ($R = n$), matching the eager join (Listing 7).*

Proof. The argument mirrors the meet. Creating m reifies no edge (Listing 8), so it costs $O(1)$, and once an element is reified it is served by the generic *find* (Listing 4) and never re-enters the traverse; so all cost is incurred by the *find* that first reifies an element. That *find* enumerates x's class in the smaller dependency (line 4), which every element join-equal to x shares, computes each member's product with two *find*s, and reifies the not-yet-reified members grouped by product, at cost $O(s\,\alpha(n))$ for a class of s elements. Enumerating a class reifies all of it, so no class is scanned twice through the same dependency and each element is enumerated at most once per dependency, i.e., at most twice; each enumeration recomputes a product with two *find*s, so the work charged to a reified element is $O(\alpha(n))$, and summing over the R of them gives $O(R\,\alpha(n))$. □

[4]The dependency whose class of x is smaller. This is just an optimization that cuts exploration.

3.4 Extension to Versioned E-Graphs

This section discusses which of the above results carry over to versioned e-graphs. An e-graph is a union-find with a *congruence* invariant: it maintains an equivalence relation closed under congruence, such that $x = y$ implies $f(x) = f(y)$ for every function f. Congruence closure is maintained by the e-graph's *rebuild* operation [Willsey et al. 2021], after a batch of unions. Accordingly, the versions of a versioned e-graph form a lattice of congruent equivalence relations, and we must reexamine whether its meet and join are still well-defined.

The meet is well-defined by the same construction as on the union-find. Starting from a trivial e-graph and replaying the dependencies' unions produces an e-graph by construction (Listing 5). However, our lazy meet algorithm (Listing 6) does not rely on the congruence-restoring union of the e-graph, so it does not preserve congruence on its own, and would require some additional machinery to restore it. The machinery mirrors the e-graph's own maintenance: whenever the lazy meet reifies a component (line 12), the e-classes of its elements are enqueued into the standard rebuild worklist, and a subsequent rebuild merges the e-nodes made congruent by the new meet edges, possibly reifying further meet edges in turn until a fixpoint. The extra cost is that of a rebuild over the reified classes only, so laziness is preserved: parts of the meet that no query touches trigger no congruence work.

The join, however, is not finitely representable in general [Gulwani et al. 2004]. The join includes all equalities implied independently by both dependencies and, unlike the meet, it admits no finite construction in general: the intersection of two finitely generated congruences need not be finitely generated. Over the signature $\{a, b, f, g\}$, let v_1 prove $a = b$ and v_2 prove $f(a) = a$, $f(b) = b$, and $g(a) = g(b)$. In v_1, congruence yields $g(f^n(a)) = g(f^n(b))$ for every n because $a = b$; in v_2 the same equalities hold because $f^n(a) = a$, $f^n(b) = b$, and $g(a) = g(b)$. Their intersection is thus the infinite family $\{\, g(f^n(a)) = g(f^n(b)) \mid n \geq 0 \,\}$, yet neither the generator $a = b$ nor the cycle $f(a) = a$ is shared, so no finite set of equalities generates it. The abstract lattice survives, since congruences over the term algebra form a complete lattice and the intersection is always a congruence; what fails is representability, as that congruence cannot be stored in an e-graph. One can still compute a *relatively complete* join, which is exact only on a chosen finite set of *important terms*, e.g., those that are already materialized in the e-graph. This is exactly what our lazy join (Listing 8) computes on the underlying union-find: the product of canonicals keeps the equalities the represented terms share. Over those terms it even yields an e-graph directly, with no rebuild. Each dependency's *find* already reflects its congruence, so the product intersects the two induced congruences, not merely the stored equalities, and an intersection of congruences is again a congruence; the result is thus congruent by construction.

4 Evaluation

We quantify, in isolation, the cost of computing the meet and the join lazily, as in Listings 6 and 8, against materializing them eagerly, as in Listings 5 and 7. Our aim is to measure both where laziness is expected to pay off, when only a fraction of a composed relation is ever queried, and its worst case, when the whole relation is forced. Below, *composition* refers to a meet or a join operation.

Implementations. Both variants share the same versioned union-find core and differ only in when a composition reifies its edges. The eager compositions materialize the full relation when the version is created, replaying the dependencies over the entire universe (Listings 5 and 7). Instead, the lazy compositions record only their dependencies at creation time and reify edges on demand during *find*, amortizing the cost across queries (Listings 6 and 8). Because the two meet and join exercise different algorithms, we evaluate them separately, so that the per-operation cost of one is not confounded by the presence of the other.

Workloads. We construct versioned union-finds with randomized sequences of operations over a universe of n elements. Each sequence first adds the n elements and then samples operations from a uniform mix of union, find, fork, and the composition under evaluation, drawn from a seeded pseudorandom generator so that every run is reproducible. Two parameters control the size of a workload: the universe size n and the number of operations. We consider two regimes. In the *average case* we sample the mix directly, so that only some canonicals of each composed version are ever queried. In the *worst case* we canonicalize every element immediately after each composition; this forces the lazy variant to materialize the entire relation with no opportunity to amortize, and is the adversarial workload on which we expect laziness to be less effective or comparable w.r.t. the eager baseline.

Metrics. We report four quantities, each collected in a separate pass over identical inputs so that measuring one never perturbs another. Two are deterministic and independent of language and machine, and we treat them as primary: the number of *reified edges*, i.e., the versioned parent pointers materialized in the structure, which measures space; and the number of *operations* performed, i.e., edge reifications and *find* traversals, which measures work. Two are empirical and corroborate the first two: wall-clock *time*, measured on the bare structure with garbage collection disabled and taken as the minimum over repeated replays to suppress noise, then reported as the median over independent trials with its interquartile range; and *peak memory*.

Scaling. We vary the size of the universe and scale the number of operations proportionally, so that the structure itself grows and the asymptotic behavior of a composition becomes visible. We plot it on a logarithmic axis and fit the empirical exponent b of a power law $cost \sim n^b$: a $b = 2$ indicates that the y-axis scales quadratically w.r.t. the x-axis. Since the number of compositions in the workload scales with the size of the universe n, we expect eager composition to show a $n \cdot O(n\alpha(n)) = O(n^2\alpha(n)) \simeq O(n^2)$, meaning $b \simeq 2$, and lazy composition to be lower in the average case or equal in the worst case, meaning $b \leq 2$.

Results. On average, the lazy meet and join reify only the edges that queries actually touch, performing up to 94% fewer operations and using up to 93% less memory than the eager baseline, and this gap widens with n. At the largest universe ($n = 800$) the meet drops from 11.62 s to 0.46 s and the join from 5.80 s to 0.48 s, while peak memory falls from roughly 587 and 596 MiB to 41 MiB in both cases. In the worst case, where every element is canonicalized after each composition, the lazy variants materialize the same relation as eager and add only a small constant overhead, at most about 12% in time and no measurable cost in memory (within 1% for $n \geq 200$). On the universe-size axis the eager cost grows as n^2 (fitted $b \simeq 2.0$), whereas the lazy cost grows as $n^{1.5}$ (fitted $b \simeq 1.5$) in the average case and converges to n^2 only in the worst case, when the full relation is forced. Figure 7 summarizes the results by plotting the number of add_edge calls, i.e., the reified edges, across the four composition/regime combinations and makes these exponents visible directly. More detailed charts are included in Section A of the appendix, corroborating the same trends in time, memory, and work.

5 Related Work

E-Graphs. E-graphs were originally developed by Nelson [1980] as a data structure for efficient congruence closure in automated theorem provers, building on the union-find algorithm of Galler and Fisher [1964]. Tarjan [1975] analyzed path compression and union-by-rank, establishing the near-optimal $O(n\,\alpha(n))$ amortized complexity for union-find [Hopcroft and Ullman 1973] that underlies all e-graph implementations including ours. Nelson and Oppen [1980] showed how congruence closure can serve as a decision procedure for equalities over uninterpreted functions,

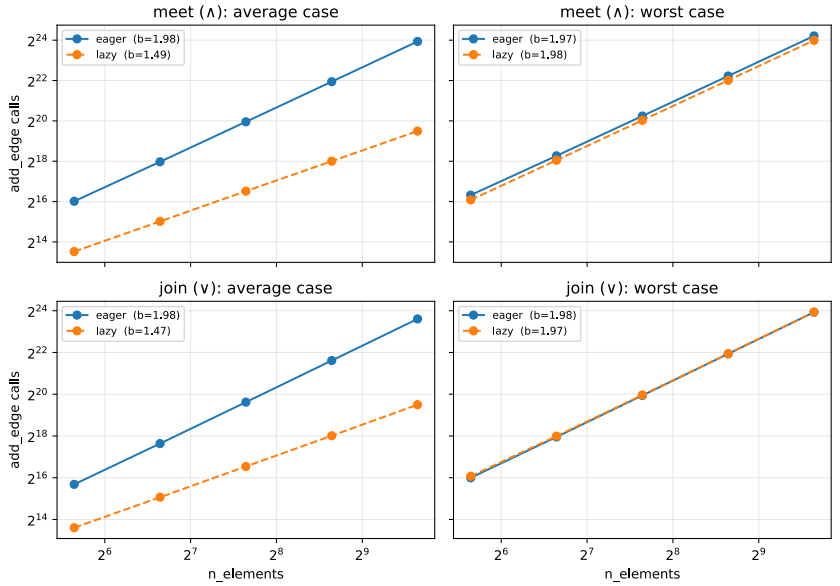


Fig. 7. Number of add_edge calls during the benchmark, i.e., the number of edges reified in the structure, as the universe size n grows (both axes logarithmic). Each column isolates one composition (meet or join). Each row isolates one regime (average or worst case scenarios). Each chart compares the eager baseline against the lazy variant, together with the fitted power law $cost \sim n^b$. Each point is the median over 100 seeds and 5 iterations per seed.

forming the basis of the theory of uninterpreted functions in SMT solvers such as Z3 [de Moura and Bjørner 2008] and cvc5 [Barbosa et al. 2022]. Simplify [Detlefs et al. 2005] pioneered e-graph-based matching inside a theorem prover; de Moura and Bjørner [2007] later gave an efficient e-matching algorithm for SMT solvers. Tate et al. [2009] introduced *equality saturation*, repurposing e-graphs for compiler optimization by applying all rewrite rules simultaneously and then extracting the optimal program, avoiding phase-ordering issues. Willsey et al. [2021] refined this approach in the egg library with e-class analyses and a rebuilding procedure for efficient invariant restoration. Applications of equality saturation include floating-point accuracy improvement [Panchekha et al. 2015], CAD model synthesis [Nandi et al. 2020], DSP vectorization [VanHattum et al. 2021], linear algebra optimization [Wang et al. 2020], tensor superoptimization [Yang et al. 2021], and library learning [Cao et al. 2023]. Proof-producing congruence closure [Nieuwenhuis and Oliveras 2005] and small proof extraction from e-graphs [Flatt et al. 2022] address the generation of compact certificates from e-graph reasoning.

E-Graph Extensions. Versioned e-graphs introduce a *fork* operation that creates a child version inheriting the parent's equivalences; each version maintains an independent view over a shared term base, and the version structure forms a tree. Our extension adds *meet* and *join* operations, turning the version tree into a version DAG and aggregating multiple versions' equivalences by union and intersection. The closest related extension is Easter Egg [Singher and Shachar 2024], which uses *colored e-graphs* to reason under multiple hypotheses simultaneously: each color corresponds to an additional assumption, and terms can be equal under one color but not another. While colored e-graphs and versioned e-graphs share the goal of maintaining separate equational contexts, colored e-graphs support only assumption addition (akin to forking) and lack

any join mechanism to compose the equivalence relations induced by those contexts, making them unable to aggregate results from independent branches. Furthermore, colored e-graphs propagate colors eagerly and do not amortize traversal across queries as lazy versioned traversal does. Zhang et al. [2022] propose relational e-matching, which recasts pattern matching against e-graphs as relational algebra queries for improved efficiency. egglog [Zhang et al. 2023] unifies e-graphs and Datalog, enabling lattice-based reasoning and incremental fixpoint computation within the equality saturation framework. Dis-equality graphs [Zakhour et al. 2024b] augment e-graphs with explicit disequality constraints, allowing the congruence closure to refute equalities in addition to establishing them. Enumo [Pal et al. 2023] supports theory exploration by providing a domain-specific language for guided rewrite rule discovery. Entangled program spaces [Koppel et al. 2022] generalize e-graphs to equality-constrained tree automata, representing program spaces in which subterm choices are not independent. Babble [Cao et al. 2023] combines e-graphs with anti-unification for library learning.

Composing Equivalence Relations. Computed eagerly, both composition operations are classical: the meet of two equivalence relations is obtained by replaying one relation's unions into the other, and the join, i.e., the coarsest common refinement of two partitions, by partition refinement [Paige and Tarjan 1987]. These algorithms are offline, processing the entire universe regardless of what is queried; our contribution is not the operations themselves but their lazy, query-driven computation inside a version DAG, amortized across queries and consistent with previously reified state. Gulwani et al. [2004] study join algorithms for congruences over uninterpreted functions in the setting of abstract interpretation, establishing the non-representability result we build on in Section 3.4; their construction is an eager product over a chosen term set, whereas our join computes the same intersection lazily on the underlying union-find. Persistent union-finds [Conchon and Filliâtre 2007] add versioning of a different kind: they let a single evolving relation be rewound and replayed, i.e., they version one relation over time, whereas versioned union-finds maintain many related relations that are simultaneously live, and this work adds the operations to compose them.

Conditional Reasoning. Case splitting is a foundational technique in both automated theorem proving and SAT/SMT solving. The DPLL(T) architecture [Nieuwenhuis et al. 2006] interleaves a SAT solver driving case splits with theory solvers that propagate equalities; the fork operation in versioned e-graphs directly models this interaction by creating a child version that inherits parent equalities and adds the case hypothesis. Conditional term rewriting [Falke and Kapur 2012; Reddy 1990] extends term rewrite systems with guards evaluated before a rule fires; unlike versioned e-graphs, which maintain multiple equational contexts simultaneously, conditional rewriting prunes branches eagerly. E-matching with guards inside SMT solvers [Niemetz et al. 2021] handles quantified formulas by instantiating patterns under particular theory assumptions, a task that versioned e-graphs can naturally support. The Propel verifier [Zakhour et al. 2023, 2024a] reasons about algebraic laws of programs by combining equality and inequality reasoning.

6 Conclusion

We revisited versioned union-finds through the lattice of equivalence relations, the equality-side presentation of the partition lattice studied by partition logic. The equivalence relations denoted by versions form a lattice under coarsening. This observation exposes two composition operations that tree-based versioning cannot express: the *meet*, which combines the equalities of several versions by the equivalence closure of their union, and the *join*, which retains the equalities common to all of them by intersection. Together they turn the version tree into a version DAG, letting a prover merge the facts discovered under independently explored assumption contexts, or extract the facts shared across the cases of a case split, without replaying work or duplicating relations. We gave

algorithms that realize both operations lazily on top of an ordinary union-find, reifying edges only as queries demand them, proved them correct, and analyzed their complexity. Our implementation confirms the analysis: the lazy meet and join scale as about $n^{1.5}$ in the size of the universe, against n^2 for their eager counterparts, and fall back to the eager n^2 behavior only in the adversarial worst case, where they add at most a small constant overhead.

Several directions remain open. Both operations extend to versioned e-graphs, where the meet preserves congruence by construction but our lazy variant would need additional machinery to restore it, and the join is only *relatively complete*, exact over a chosen set of important terms, since the intersection of two finitely generated congruences need not itself be finitely generated. The independence of the composed versions also makes the structure a natural fit for parallel proof search: forks introduce case hypotheses that can be saturated concurrently, and a subsequent meet or join aggregates their results, as in cube-and-conquer. Finally, integrating the version DAG into a prover or SMT solver and measuring its effect on end-to-end proof search is the natural next step toward putting these operations to work.

References

Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems: 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2–7, 2022, Proceedings, Part I* (Munich, Germany). Springer-Verlag, Berlin/Heidelberg, Germany, 415–442. https://doi.org/10.1007/978-3-030-99524-9_24

David Cao, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. babble: Learning Better Abstractions with E-Graphs and Anti-unification. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 14 (Jan. 2023), 29 pages. https://doi.org/10.1145/3571207

Jahrim Gabriele Cesario, George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. 2026. Versioned E-Graphs. *Proceedings of the ACM on Programming Languages* 10, PLDI, Article 171 (June 2026), 23 pages. https://doi.org/10.1145/3808249

Sylvain Conchon and Jean-Christophe Filliâtre. 2007. A Persistent Union-Find Data Structure. In *Proceedings of the 2007 Workshop on ML* (Freiburg, Germany) *(ML '07)*. Association for Computing Machinery, New York, NY, USA, 37–46. https://doi.org/10.1145/1292535.1292541

Samuel Coward, George A. Constantinides, and Theo Drane. 2023. Automating Constraint-Aware Datapath Optimization using E-Graphs. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC56929.2023.10247797

Leonardo de Moura and Nikolaj Bjørner. 2007. Efficient E-Matching for SMT Solvers. In *s* (Bremen, Germany) *(CADE '21)*, Frank Pfenning (Ed.). Springer-Verlag, Berlin, Heidelberg, 183–198. https://doi.org/10.1007/978-3-540-73595-3_13

Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (Budapest, Hungary) *(TACAS '08)*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer-Verlag, Berlin/Heidelberg, Germany, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

David Detlefs, Greg Nelson, and James B. Saxe. 2005. Simplify: A Theorem Prover for Program Checking. *J. ACM* 52, 3 (May 2005), 365–473. https://doi.org/10.1145/1066100.1066102

David Ellerman. 2010. The Logic of Partitions: Introduction to the Dual of the Logic of Subsets. *The Review of Symbolic Logic* 3, 2 (2010), 287–350. https://doi.org/10.1017/S1755020310000018

Stephan Falke and Deepak Kapur. 2012. Rewriting Induction + Linear Arithmetic = Decision Procedure. In *Automated Reasoning (Lecture Notes in Computer Science)*, Bernhard Gramlich, Dale Miller, and Uli Sattler (Eds.). Springer-Verlag, Berlin/Heidelberg, Germany, 241–255. https://doi.org/10.1007/978-3-642-31365-3_20

Oliver Flatt, Samuel Coward, Max Willsey, Zachary Tatlock, and Pavel Panchekha. 2022. Small Proofs from Congruence Closure. In *Proceedings of the 22nd Conference on Formal Methods in Computer-Aided Design* (Trento, Italy) *(FMCAD '24)*, Alberto Griggio and Neha Rungta (Eds.). TU Wien Academic Press, 75–83. https://doi.org/10.34727/2022/isbn.978-3-85448-053-2_13

Bernard A. Galler and Michael J. Fisher. 1964. An Improved Equivalence Algorithm. *Commun. ACM* 7, 5 (May 1964), 301–303. https://doi.org/10.1145/364099.364331

George Grätzer. 2011. *Lattice Theory: Foundation*. Birkhäuser Basel. https://doi.org/10.1007/978-3-0348-0018-1

Sumit Gulwani, Ashish Tiwari, and George C. Necula. 2004. Join Algorithms for the Theory of Uninterpreted Functions. In *Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2004) (Lecture Notes in Computer Science, Vol. 3328)*. Springer, 311–323. https://doi.org/10.1007/978-3-540-30538-5_26

J. E. Hopcroft and J. D. Ullman. 1973. Set Merging Algorithms. *SIAM J. Comput.* 2, 4 (Dec. 1973), 294–303. https://doi.org/10.1137/0202024

James Koppel, Zheng Guo, Edsko de Vries, Armando Solar-Lezama, and Nadia Polikarpova. 2022. Searching Entangled Program Spaces. *Proceedings of the ACM on Programming Languages* 6, ICFP, Article 91 (Aug. 2022), 29 pages. https://doi.org/10.1145/3547622

Cole Kurashige, Ruyi Ji, Aditya Giridharan, Mark Barbone, Daniel Noor, Shachar Itzhaky, Ranjit Jhala, and Nadia Polikarpova. 2024. CCLemma: E-Graph Guided Lemma Discovery for Inductive Equational Proofs. *Proceedings of the ACM on Programming Languages* 8, ICFP, Article 264 (Aug. 2024), 27 pages. https://doi.org/10.1145/3674653

Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs Using Linear Ghost Types. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1, Article 85 (2023). https://doi.org/10.1145/3586037

K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16) (Lecture Notes in Computer Science, Vol. 6355)*. Springer, Berlin, Heidelberg, 348–370. https://doi.org/10.1007/978-3-642-17511-4_20

Chandrakana Nandi, Max Willsey, Adam Anderson, James R. Wilcox, Eva Darulova, Dan Grossman, and Zachary Tatlock. 2020. Synthesizing structured CAD models with equality saturation and inverse transformations. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) *(PLDI '20)*. ACM, New York, NY, USA, 31–44. https://doi.org/10.1145/3385412.3386012

Charles Gregory Nelson. 1980. *Techniques for Program Verification.* Ph. D. Dissertation. Stanford, CA, USA. AAI8011683.

Greg Nelson and Derek C. Oppen. 1980. Fast Decision Procedures Based on Congruence Closure. *J. ACM* 27, 2 (April 1980), 356–364. https://doi.org/10.1145/357933.357941

Aina Niemetz, Mathias Preiner, Andrew Reynolds, Clark Barrett, and Cesare Tinelli. 2021. Syntax-Guided Quantifier Instantiation. In *Tools and Algorithms for the Construction and Analysis of Systems* (Luxembourg, Luxembourg), Jan Friso Groote and Kim Guldstrand Larsen (Eds.). Springer-Verlag, Berlin, Heidelberg, 145–163. https://doi.org/10.1007/978-3-030-72013-1_8

Robert Nieuwenhuis and Albert Oliveras. 2005. Proof-Producing Congruence Closure. In *Proceedings of the 16th International Conference on Term Rewriting and Applications* (Nara, Japan) *(RTA'05)*. Springer-Verlag, Berlin, Heidelberg, 453–468. https://doi.org/10.1007/978-3-540-32033-3_33

Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. 2006. Solving SAT and SAT Modulo Theories: From an Abstract Davis–Putnam–Logemann–Loveland Procedure to DPLL(T). *J. ACM* 53, 6, 937–977. https://doi.org/10.1145/1217856.1217859

Oystein Ore. 1942. Theory of equivalence relations. *Duke Mathematical Journal* 9, 3 (1942), 573 – 627. https://doi.org/10.1215/S0012-7094-42-00942-6

Robert Paige and Robert E. Tarjan. 1987. Three Partition Refinement Algorithms. *SIAM J. Comput.* 16, 6 (1987), 973–989. https://doi.org/10.1137/0216062

Anjali Pal, Brett Saiki, Ryan Tjoa, Cynthia Richey, Amy Zhu, Oliver Flatt, Max Willsey, Zachary Tatlock, and Chandrakana Nandi. 2023. Equality Saturation Theory Exploration à la Carte. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2, Article 258 (Oct. 2023), 29 pages. https://doi.org/10.1145/3622834

Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. 2015. Automatically improving accuracy for floating point expressions. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) *(PLDI '15)*. ACM, New York, NY, USA, 1–11. https://doi.org/10.1145/2737924.2737959

Uday S. Reddy. 1990. Term Rewriting Induction. In *10th International Conference on Automated Deduction (Lecture Notes in Computer Science)*, Mark E. Stickel (Ed.). Springer-Verlag, Berlin/Heidelberg, Germany, 162–177. https://doi.org/10.1007/3-540-52885-7_86

Eytan Singher and Shachar Itzhaky. 2021. Theory Exploration Powered by Deductive Synthesis. In *Computer Aided Verification*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer International Publishing, Cham, Switzerland, 125–148. https://doi.org/10.1007/978-3-030-81688-9_6

Eytan Singher and Itzhaky Shachar. 2024. Easter Egg: Equality Reasoning Based on E-Graphs with Multiple Assumptions. In *Proceedings of the 24th Conference on Formal Methods in Computer-Aided Design* (Prague, Czech Republic) *(FMCAD '24)*, Nina Narodytska and Philipp Rümmer (Eds.). TU Wien Academic Press, Wien, Austria, 70–83. https://doi.org/10.34727/2024/isbn.978-3-85448-065-5_13

Robert Endre Tarjan. 1975. Efficiency of a Good But Not Linear Set Union Algorithm. *J. ACM* 22, 2 (April 1975), 215–225. https://doi.org/10.1145/321879.321884

Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality Saturation: A New Approach to Optimization. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Savannah, GA, USA) *(POPL '09)*. ACM, New York, NY, USA, 264–276. https://doi.org/10.1145/1480881.1480915

The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs* (New Orleans, LA, USA) *(CPP 2020)*. Association for Computing Machinery, New York, NY, USA, 367–381. https://doi.org/10.1145/3372885.3373824

Alexa VanHattum, Rachit Nigam, Vincent T. Lee, James Bornholt, and Adrian Sampson. 2021. Vectorization for Digital Signal Processors via Equality Saturation. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. ACM, New York, NY, USA, 874–886. https://doi.org/10.1145/3445814.3446707

Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: Sum-Product Optimization via Relational Equality Saturation for Large Scale Linear Algebra. *Proceedings of the VLDB Endowment* 13, 12 (July 2020), 1919–1932. https://doi.org/10.14778/3407790.3407799

Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and Extensible Equality Saturation. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 23 (Jan. 2021), 29 pages. https://doi.org/10.1145/3434304

Yichen Yang, Phitchaya Mangpo Phothilimthana, Yisu Remy Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. 2021. Equality Saturation for Tensor Graph Superoptimization. In *Proceedings of the 4th Conference on Machine Learning and Systems (MLSys '21)*, Alex Smola, Alex Dimakis, and Ion Stoica (Eds.). MLSys.

George Zakhour, Pascal Weisenburger, Jahrim Gabriele Cesario, and Guido Salvaneschi. 2024b. *Dis/Equality Graphs*. https://doi.org/10.5281/zenodo.13938878

George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. 2023. Type-Checking CRDT Convergence. *Proceedings of the ACM on Programming Languages* 7, PLDI, Article 162 (June 2023), 24 pages. https://doi.org/10.1145/3591276

George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. 2024a. Automated Verification of Fundamental Algebraic Laws. *Proceedings of the ACM on Programming Languages* 8, PLDI, Article 178 (June 2024), 24 pages. https://doi.org/10.1145/3656408

Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better Together: Unifying Datalog and Equality Saturation. *Proceedings of the ACM on Programming Languages* 7, PLDI, Article 125 (June 2023), 25 pages. https://doi.org/10.1145/3591239

Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2022. Relational E-Matching. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 35 (Jan. 2022), 22 pages. https://doi.org/10.1145/3498696

A Detailed Evaluation Results

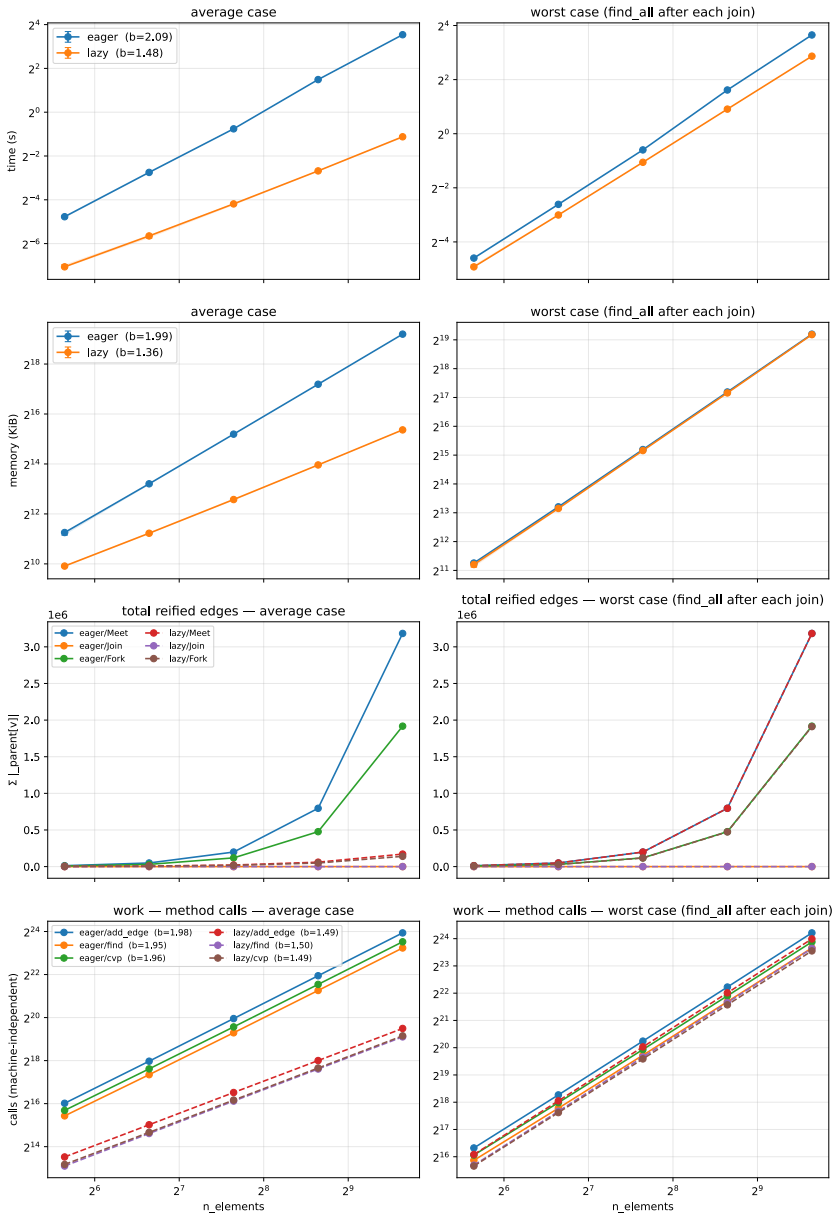


Fig. 8. Scaling of the eager and lazy meet as the universe size n grows (both axes logarithmic). Rows, top to bottom: wall-clock *time* (s); peak *memory* (KiB); the number of *reified edges*, i.e., the versioned parent pointers materialized in the structure ($\sum_v |\_\texttt{parent}[v]|$); and machine-independent *operations*, i.e., add_edge, *find*, and cvp calls. The left column is the average case; the right column is the worst case, which canonicalizes every element after each meet. Solid lines are eager, dashed lines are lazy, and each legend entry reports the fitted power-law exponent b of $cost \sim n^b$. The eager meet scales roughly quadratically ($b \approx 2.0$ to 2.1 in time), whereas the lazy meet stays near $b \approx 1.5$, so its advantage grows with n.

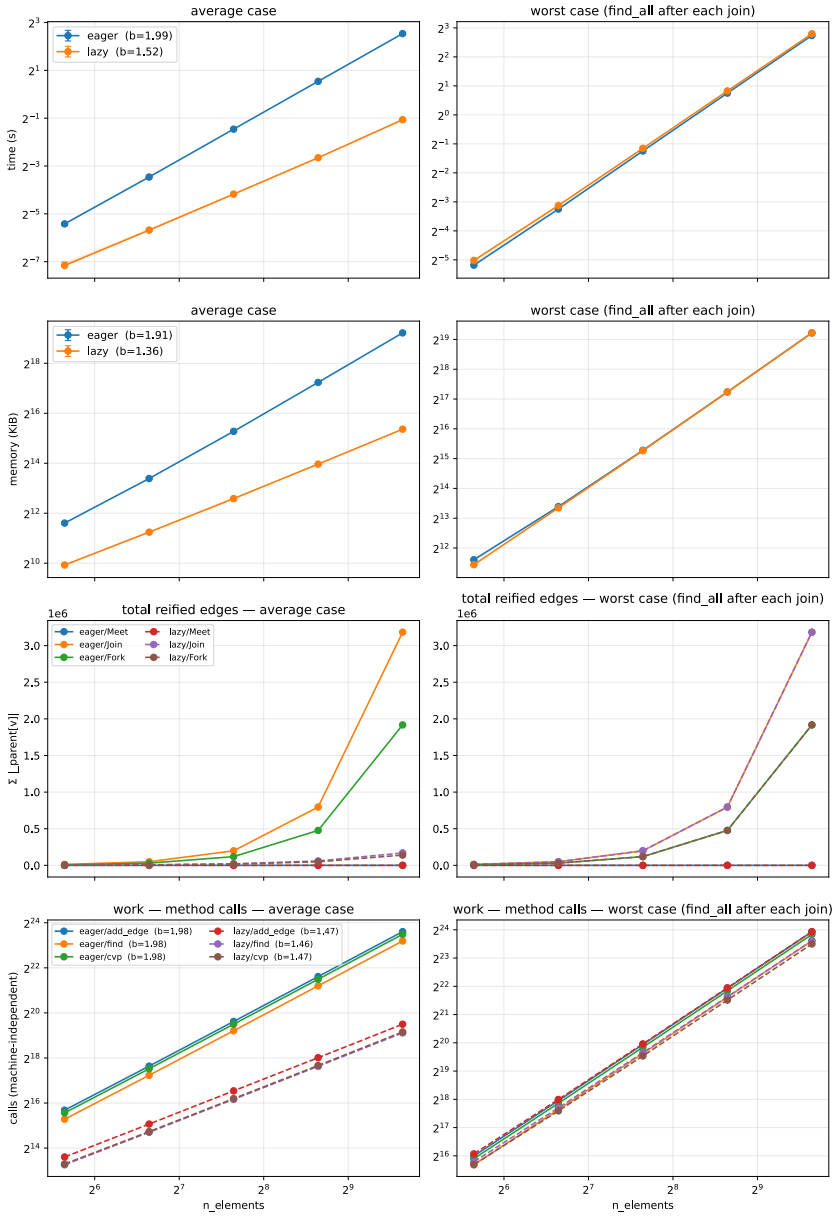


Fig. 9. Scaling of the eager and lazy join as the universe size n grows (both axes logarithmic). Rows, top to bottom: wall-clock *time* (s); peak *memory* (KiB); the number of *reified edges*, i.e., the versioned parent pointers materialized in the structure ($\sum_v |\_\mathrm{parent}[v]|$); and machine-independent *operations*, i.e., add_edge, *find*, and cvp calls. The left column is the average case; the right column is the worst case, which canonicalizes every element after each join. Solid lines are eager, dashed lines are lazy, and each legend entry reports the fitted power-law exponent b of *cost* $\sim n^b$. The lazy join grows with a smaller exponent (about 1.5 in time and work, 1.4 in memory) than the eager join (about 2.0), so its margin widens as n increases.